SESSION 9

ADVANCED JOINS

FULL OUTER JOIN • SELF JOIN • CROSS JOIN

Goal: Solve all five tasks with MySQL-compatible SQL, grey code boxes, and simple explanations.

MySQL note: MySQL does not directly support FULL OUTER JOIN. We can reproduce it with a LEFT JOIN plus unmatched rows from a RIGHT JOIN, combined with UNION ALL.

Task 1 — FULL OUTER JOIN: Influencers and Brands

Create two tables with at least three rows each. Match influencers and brands by city. The result should keep matches and also show unmatched influencers or brands, using NULL for the missing side.

Step A: Create tables and insert sample rows

```
CREATE DATABASE IF NOT EXISTS join_practice;
USE join_practice;

CREATE TABLE influencers (
    influencer_id INT PRIMARY KEY,
    influencer_name VARCHAR(100),
    city VARCHAR(50)
);

CREATE TABLE brands (
    brand_id INT PRIMARY KEY,
    brand_name VARCHAR(100),
    city VARCHAR(50)
);

INSERT INTO influencers (influencer_id, influencer_name, city)
VALUES
    (1, 'Aisha Verma', 'Mumbai'),
    (2, 'Rohan Mehta', 'Delhi'),
    (3, 'Neha Shah', 'Pune');

INSERT INTO brands (brand_id, brand_name, city)
VALUES
    (101, 'GlowUp', 'Mumbai'),
    (102, 'UrbanLeaf', 'Delhi'),
    (103, 'TrendNest', 'Bengaluru');
```

The first two cities match: Mumbai and Delhi. Neha Shah is in Pune without a matching brand, while TrendNest is in Bengaluru without a matching influencer.

Step B: Emulate FULL OUTER JOIN in MySQL

```
SELECT
    i.influencer_name,
    b.brand_name,
    i.city AS influencer_city,
    b.city AS brand_city
FROM influencers AS i
LEFT JOIN brands AS b
    ON i.city = b.city

UNION ALL

SELECT
    i.influencer_name,
    b.brand_name,
    i.city AS influencer_city,
    b.city AS brand_city
FROM influencers AS i
RIGHT JOIN brands AS b
    ON i.city = b.city
WHERE i.influencer_id IS NULL;
```

How it works: LEFT JOIN keeps every influencer, even if no brand matches. The RIGHT JOIN section finds brands without a matching influencer. The WHERE clause keeps only those unmatched brand rows. UNION ALL combines both results without dropping any rows.

influencer_name	brand_name	influencer_city	brand_city
Aisha Verma	GlowUp	Mumbai	Mumbai
Rohan Mehta	UrbanLeaf	Delhi	Delhi
Neha Shah	NULL	Pune	NULL
NULL	TrendNest	NULL	Bengaluru

Remember: NULL appears where there is no matching row on that side of the join.

Task 2 — SELF JOIN: Playlist and Parent Playlist

A SELF JOIN joins a table to itself. In this example, one copy of playlists represents the child playlist and another copy represents its parent.

```
SELECT
    child.id,
    child.playlist_name,
    child.parent_playlist_id,
    parent.playlist_name AS parent_playlist_name
FROM playlists AS child
LEFT JOIN playlists AS parent
    ON child.parent_playlist_id = parent.id;
```

Explanation

- `child` is the alias for the playlist row being displayed.
- `parent` is another alias for the same playlists table, used to find the parent playlist.
- `child.parent_playlist_id = parent.id` matches each child to its parent.
- `LEFT JOIN` keeps playlists that do not have a parent. Their `parent_playlist_name` will be NULL.

Key idea: SELF JOIN is useful when rows in one table refer to other rows in the same table, such as nested playlists or employee-manager relationships.

Task 3 — CROSS JOIN: Users and Offers

A CROSS JOIN creates every possible combination of rows from the users table and the offers table.

Step A: Create tables and insert sample rows

```
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    user_name VARCHAR(100),
    city VARCHAR(50)
);

CREATE TABLE offers (
    offer_id INT PRIMARY KEY,
    offer_title VARCHAR(100)
);

INSERT INTO users (user_id, user_name, city)
VALUES
    (1, 'Anaya', 'Mumbai'),
    (2, 'Kabir', 'Delhi'),
    (3, 'Meera', 'Mumbai');

INSERT INTO offers (offer_id, offer_title)
VALUES
    (101, '10% OFF'),
    (102, 'Free Delivery'),
    (103, 'Weekend Deal');
```

Step B: Generate all user-offer combinations

```
SELECT
    u.user_name,
    o.offer_title
FROM users AS u
CROSS JOIN offers AS o;

-- Campaign idea: create every user-offer pairing first,
-- then use customer preferences, location, or purchase history
-- to select suitable offers for a Flipkart-style campaign.
```

With 3 users and 3 offers, the query returns 3 × 3 = 9 rows. Each user is paired with each offer. The comment describes one possible use: generate candidate pairings, then personalize or filter the offers using customer data.

Remember: CROSS JOIN does not require an ON condition. If the first table has m rows and the second has n rows, the result has m × n combinations.

Task 4 — SELF JOIN: Employees and Managers

Assume employees has the columns id, name, and manager_id. The manager_id value points to another employee's id. A NULL manager_id means the employee does not have a manager.

Step A: Display every employee with their manager

```
SELECT
    e.name AS employee_name,
    m.name AS manager_name
FROM employees AS e
LEFT JOIN employees AS m
    ON e.manager_id = m.id;
```

The first alias (`e`) represents the employee. The second alias (`m`) represents the manager. The join connects an employee's manager_id to the manager's id. LEFT JOIN keeps employees even when manager_id is NULL.

Step B: Show only employees who do not have a manager

```
SELECT
    e.name AS employee_name,
    m.name AS manager_name
FROM employees AS e
LEFT JOIN employees AS m
    ON e.manager_id = m.id
WHERE e.manager_id IS NULL;
```

The condition WHERE e.manager_id IS NULL filters the result to employees who have no manager recorded. Their manager_name is also NULL. These are the top-level employees in the hierarchy.

Task 5 — SELF JOIN: Find User Pairs in the Same City

Assume the users table has user_id, user_name, and city. This query finds pairs of different users who live in the same city.

```
SELECT
    u1.user_id,
    u1.user_name,
    u1.city,
    u2.user_id AS matched_user_id,
    u2.user_name AS matched_user_name
FROM users AS u1
JOIN users AS u2
    ON u1.city = u2.city
   AND u1.user_id < u2.user_id;
```

Explanation

- `u1` and `u2` are two aliases for the same users table.
- `u1.city = u2.city` matches users who live in the same city.
- `u1.user_id < u2.user_id` prevents a user from being paired with themselves and avoids displaying the same pair twice in reverse order.

Brief description of how AI helped

AI helped structure the SELF JOIN using two aliases for the users table, match the city values, and add the condition `u1.user_id < u2.user_id`. That condition removes self-comparisons and prevents duplicate reversed pairs, making the results easier to read.

Example: If Anaya and Meera both live in Mumbai, the result contains the pair Anaya–Meera once, not Anaya–Anaya, Meera–Meera, or both Anaya–Meera and Meera–Anaya.

Quick Revision — When to Use Each Join

Join type	What it does	Typical use
FULL OUTER JOIN	Keeps matching and non-matching rows from both tables.	Find all influencers and brands, including unmatched records.
SELF JOIN	Joins a table to itself using different aliases.	Employee-manager or child-parent relationships; same-city user pairs.
CROSS JOIN	Returns every possible combination between two tables.	Generate candidate user-offer pairs before campaign filtering.
LEFT JOIN	Keeps all rows from the left table, with matches from the right.	Keep influencers or employees even when no match exists.
RIGHT JOIN	Keeps all rows from the right table, with matches from the left.	Find rows such as brands that have no matching influencer.
UNION ALL	Combines result sets without removing duplicate rows.	Combine the left-join results with unmatched right-side rows.

MySQL note: To emulate a FULL OUTER JOIN, use LEFT JOIN plus the unmatched rows from a RIGHT JOIN. The examples above use matching city values and use UNION ALL so legitimate matched rows are not removed by duplicate elimination.